

HƯỚNG DẪN TỰ HỌC

Supervised Learning

Introduction & Model Evaluation

Machine Learning in Data Science

MỤC TIÊU HỌC TẬP

- Phân biệt Regression và Classification
- Hiểu và thực hiện đúng quy trình Supervised Learning từ đầu đến cuối
- Lựa chọn và giải thích đúng các metric đánh giá mô hình
- Nhận biết và xử lý Overfitting / Underfitting
- Tự xây dựng một mô hình hoàn chỉnh và kiểm chứng kết quả

PHẦN 1 – NỀN TẢNG LÝ THUYẾT

Bước 1 – Supervised Learning là gì

Supervised Learning = học từ dữ liệu có nhãn (labeled data). Mỗi mẫu gồm features X (đầu vào) và label y (đầu ra đúng). Mục tiêu là học ánh xạ $f(X) \rightarrow y$ để dự đoán trên dữ liệu mới.

Tự kiểm tra – Trả lời không nhìn slide:

- Supervised Learning khác Unsupervised Learning ở điểm nào?
- Cho ví dụ thực tế: 3 bài toán Regression và 3 bài toán Classification.
- Dữ liệu "labeled" nghĩa là gì? Ai tạo ra label?

Bước 2 – Regression vs Classification

Tiêu chí	Regression	Classification
Đầu ra	Số thực liên tục	Nhãn rời rạc (class)
Câu hỏi	How much? How many?	Which class?
Ví dụ	Giá nhà, nhiệt độ	Spam/ham, chữ số MNIST
Loss fn	MSE, MAE, RMSE	Cross-entropy, Hinge loss
Thuật toán	Linear/Polynomial/Lasso Reg	kNN, SVM, Decision Tree, Neural Net

Bài tập: Xác định Regression hay Classification cho mỗi bài toán sau và giải thích lý do:

1. Dự đoán số lượt xem của một video YouTube
2. Phân loại email là spam hay không
3. Dự đoán điểm thi của sinh viên
4. Nhận diện cảm xúc trong ảnh (vui/buồn/tức/ngạc nhiên)
5. Dự báo doanh thu quý tới

Bước 3 – Quy trình 7 giai đoạn

Thu thập dữ liệu → EDA → Tiền xử lý → Chia tập → Huấn luyện → Tinh chỉnh
→ Đánh giá → Triển khai

#	Giai đoạn	Mô tả
1	Data Collection	Thu thập dữ liệu có nhãn: chọn features, đảm bảo label đúng
2	EDA	Khám phá dữ liệu: histogram, boxplot, scatter plot, ma trận tương quan
3	Preprocessing	Xử lý missing values, chuẩn hóa dữ liệu (scaling)
4	Train/Test Split	Chia TRƯỞC khi preprocessing để tránh data leakage
5	Model Training	Chọn thuật toán, gọi .fit() trên tập train

6	Fine Tuning	Tối ưu hyperparameter với Grid Search + Cross-Validation
7	Evaluation	Đánh giá trên test set – chỉ dùng 1 lần duy nhất!

⚠️ QUY TẮC VÀNG: Luôn split dataset TRƯỚC khi preprocessing để tránh data leakage!

Tự kiểm tra:

- Tại sao phải chia train/test TRƯỚC khi chuẩn hóa dữ liệu?
- Validation set khác Test set ở chỗ nào?
- Cross-validation giải quyết vấn đề gì mà single split không làm được?

Bước 4 – Train/Test Split

Random Split – dùng khi dataset đủ lớn

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% test
    random_state=42         # đảm bảo tái lập kết quả
)
```

Stratified Split – dùng khi các class không cân bằng

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y              # giữ tỉ lệ class như gốc
)
```

PHẦN 2 – CÁC METRICS ĐÁNH GIÁ

Bước 5 – Metrics cho Regression

Metric	Ý nghĩa	Đặc điểm
MAE	Trung bình sai số tuyệt đối	Dễ hiểu, không phạt nặng outlier
MSE	Trung bình bình phương sai số	Phạt nặng sai số lớn
RMSE	$\sqrt{\text{MSE}}$	Cùng đơn vị với y, phổ biến nhất
R^2	Tỉ lệ phương sai được giải thích	1.0 = hoàn hảo; 0 = không tốt hơn baseline
MAPE	Sai số phần trăm trung bình	Dễ diễn giải, lỗi khi $y \approx 0$

 **Tự kiểm tra:** Một mô hình dự đoán giá nhà có $\text{RMSE} = 50$ triệu đồng và $R^2 = 0.85$. Mô hình này tốt không? Giải thích.

Bước 6 – Metrics cho Classification

6a. Confusion Matrix

	Predicted: Positive	Predicted: Negative
Actual: Positive	TP – True Positive ✓	FN – False Negative (Type II)
Actual: Negative	FP – False Positive (Type I)	TN – True Negative ✓

6b. Các metric từ Confusion Matrix

Metric	Công thức	Ý nghĩa
Accuracy	$(\text{TP} + \text{TN}) / \text{Total}$	Tỉ lệ đúng tổng thể
Precision	$\text{TP} / (\text{TP} + \text{FP})$	Trong số dự đoán Positive, bao nhiêu % đúng?
Recall	$\text{TP} / (\text{TP} + \text{FN})$	Trong số Positive thực tế, tìm được bao nhiêu %?
F1-Score	$2 \times \text{P} \times \text{R} / (\text{P} + \text{R})$	Cân bằng giữa Precision và Recall

6c. Khi nào dùng metric nào?

- Dataset cân bằng → Accuracy
- Tránh False Positive quan trọng hơn → Precision (ví dụ: spam filter)
- Tránh False Negative quan trọng hơn → Recall (ví dụ: phát hiện bệnh ung thư)
- Cần cân bằng cả hai → F1-Score
- Dataset mất cân bằng → F1 hoặc PR Curve, KHÔNG dùng Accuracy

 **Tự kiểm tra – Gian lận thẻ tín dụng (1% gian lận, 99% bình thường):** 1. Nếu model luôn dự đoán "bình thường", Accuracy là bao nhiêu? 2. Tại sao Accuracy không phù hợp ở đây? 3. Bạn sẽ dùng metric nào và tại sao?

Bước 7 – Overfitting & Underfitting

	Underfitting	Good Fit ✓	Overfitting
Training error	Cao	Thấp	Rất thấp
Test error	Cao	Thấp	Cao
Bias	Cao	Thấp	Thấp
Variance	Thấp	Thấp	Cao
Model	Quá đơn giản	Vừa	Quá phức tạp

K-fold Cross-Validation

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X_train, y_train, cv=5, scoring='f1')
print(f'F1 trung bình: {scores.mean():.3f} +/- {scores.std():.3f}')
# Neu std cao (>0.05) → mô hình không ổn định
```

PHẦN 3 – THỰC HÀNH: XÂY DỰNG MÔ HÌNH HOÀN CHỈNH

Dataset: Wine Quality (sklearn built-in – không cần tải về) | Thuật toán: K-Nearest Neighbors (KNN)

Bước 1 – Chuẩn bị môi trường

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (confusion_matrix, classification_report,
                             precision_score, recall_score, f1_score, ConfusionMatrixDisplay)
```

Bước 2 – Tải & EDA

```
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target, name='class')

print(f'Kích thước: {X.shape}')
print(f'Phân phối class:\n{y.value_counts()}')
print(f'Missing values: {X.isnull().sum().sum()}')

# Visualize phân phối class
y.value_counts().plot(kind='bar')
plt.title('Phân phối các class'); plt.show()

# Heatmap tương quan
sns.heatmap(X.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.show()
```

Bước 3 – Chia tập (TRƯỚC preprocessing)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f'Train: {X_train.shape[0]} mẫu | Test: {X_test.shape[0]} mẫu')
```

Bước 4 – Preprocessing (Scaling)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform trên train
X_test_scaled = scaler.transform(X_test)      # chỉ transform trên test!
# KHÔNG fit trên test → tránh data leakage
```

Bước 5 – Huấn luyện mô hình ban đầu

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

y_pred_initial = knn.predict(X_test_scaled)
print(classification_report(y_test, y_pred_initial,
                           target_names=wine.target_names))
```

Bước 6 – Cross-Validation

```
cv_scores = cross_val_score(
    KNeighborsClassifier(n_neighbors=5),
    X_train_scaled, y_train,
    cv=5, scoring='f1_weighted'
)
print(f'F1 trung bình: {cv_scores.mean():.3f}')
print(f'Do lech chuan: {cv_scores.std():.3f}')
# std > 0.05 → canh bao mo hinh khong on dinh
```

Bước 7 – Grid Search (Tinh chỉnh Hyperparameter)

```
param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 15, 21],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

grid_search = GridSearchCV(
    KNeighborsClassifier(), param_grid,
    cv=5, scoring='f1_weighted', n_jobs=-1
)
grid_search.fit(X_train_scaled, y_train)

print(f'Hyperparameter tot nhat: {grid_search.best_params_}')
print(f'F1 tot nhat (CV): {grid_search.best_score_:.3f}')
best_knn = grid_search.best_estimator_
```

Bước 8 – Đánh giá cuối cùng trên Test Set

```
# Chi dung test set MOT LAN DUY NHAT
y_pred_best = best_knn.predict(X_test_scaled)
print(classification_report(y_test, y_pred_best,
                           target_names=wine.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_best)
disp = ConfusionMatrixDisplay(cm, display_labels=wine.target_names)
disp.plot(cmap='Blues'); plt.show()
```

Bước 9 – So sánh trước/sau tinh chỉnh

```
for avg in ['macro', 'weighted']:
    f1_before = f1_score(y_test, y_pred_initial, average=avg)
    f1_after = f1_score(y_test, y_pred_best, average=avg)
    print(f'F1 ({avg}): {f1_before:.3f} → {f1_after:.3f}')
```

Bước 10 – Kiểm tra Overfitting/Underfitting

```
y_pred_train = best_knn.predict(X_train_scaled)
f1_train = f1_score(y_train, y_pred_train, average='weighted')
f1_test = f1_score(y_test, y_pred_best, average='weighted')
gap = f1_train - f1_test

if f1_train < 0.7:
    print('Underfitting: model qua don gian')
elif gap > 0.1:
    print('Overfitting: model qua phuc tap')
else:
    print('Good Fit: mo hinh can bang ✓')
```

PHẦN 4 – KIỂM CHỨNG KẾT QUẢ

Checklist tự đánh giá

Về quy trình

- ☐ Tôi đã split TRƯỚC khi scale chưa?
- ☐ Scaler có chỉ .fit() trên train không?
- ☐ Test set có chỉ được dùng đúng một lần không?
- ☐ Cross-validation có được thực hiện trên train set không?

Về kết quả

- ☐ F1 trên train và test có hợp lý không (>0.6)?
- ☐ Gap giữa train và test có quá lớn không (>0.1)?
- ☐ Confusion matrix có cho thấy class nào bị nhầm nhiều không?
- ☐ Hyperparameter tối ưu có cải thiện đáng kể so với mặc định không?

Về hiểu biết

- ☐ Tôi có thể giải thích vì sao chọn metric này thay vì metric khác?
- ☐ Tôi có thể giải thích từng ô trong Confusion Matrix?
- ☐ Tôi biết phải làm gì nếu kết quả bị Overfitting?

Kết quả kỳ vọng (Wine Dataset)

Tiêu chí	KNN mặc định (k=5)	Sau Grid Search
F1-Score	~ 0.93 – 0.97	~ 0.95 – 0.98
Gap train–test	< 0.05	< 0.05 (Good Fit)

Nếu kết quả thấp hơn nhiều, hãy kiểm tra: 1. Có scale features trước khi dùng KNN không? 2. stratify=y có được dùng không? 3. Scaler có bị fit trên test set không (data leakage)?